

Variance Reduction for Training Neural Bayes Estimators

Kai Marns-Morris

Honours (BPhil) 2026

The University of Western Australia

Supervisors: Dr Michael Bertolacci & A/Prof Edward Cripps

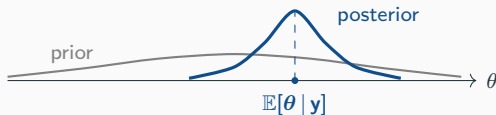
Background: neural Bayes estimators

Bayesian inference

- Fix a model: parameters θ with prior $p(\theta)$, and data $\mathbf{y}$ with likelihood $p(\mathbf{y} | \theta)$.
- Seeing $\mathbf{y}$, Bayes' rule updates the prior into the **posterior distribution**:

$$p(\theta | \mathbf{y}) \propto p(\mathbf{y} | \theta) p(\theta).$$

- The posterior is our full, updated belief about θ .

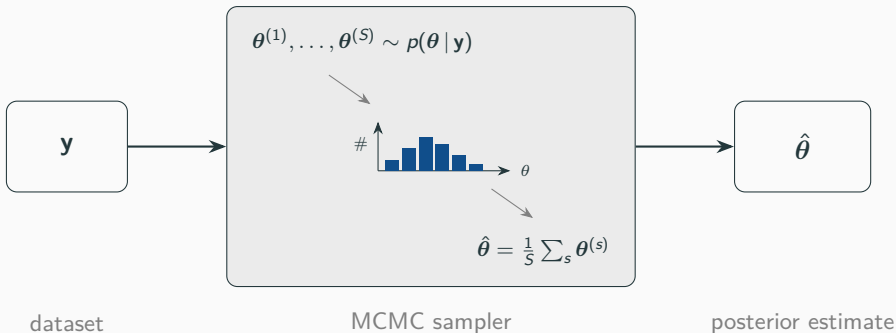


Point estimation

- Often we do not want the whole posterior — just a single **point estimate** $\hat{\theta}$ of θ .
- The usual choice is the **posterior mean** $\hat{\theta} = \mathbb{E}[\theta | \mathbf{y}]$.
- But it is **rarely available in closed form** — so we need a **procedure** that turns $\mathbf{y}$ into $\hat{\theta}$:

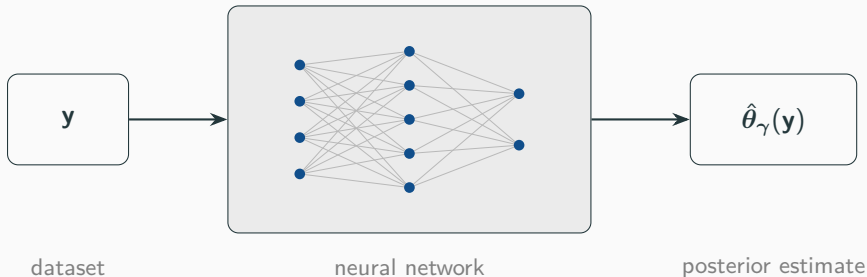


Method (1): Markov chain Monte Carlo



- Accurate and general — but must be **re-run from scratch for every new y** , and can be slow.
- Do inference for *many* datasets and you pay that cost *every time*.

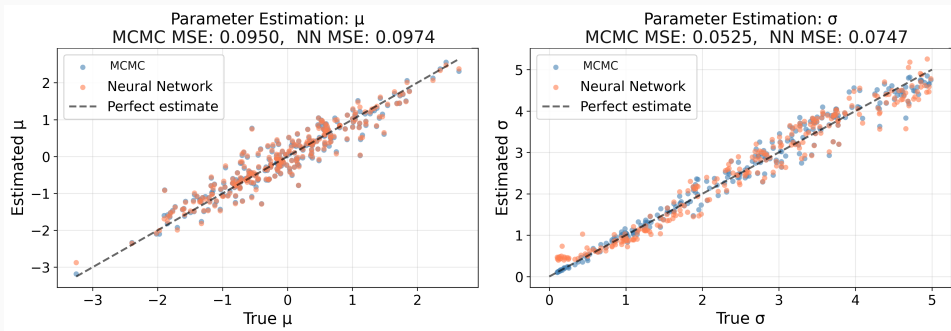
Method (2): Neural Bayes estimator



- A **neural network** $\hat{\theta}_{\gamma}(y)$, trained *once* — a *neural Bayes estimator*.
- Training data is free: **simulate** pairs (θ, y) and adjust γ so $\hat{\theta}_{\gamma}(y) \approx \theta$.
- Then each new dataset is a single **forward pass** — the cost is **amortised**.

Does it work?

A simple test: $y_1, \dots, y_n \sim \mathcal{N}(\mu, \sigma^2)$ with $\mu \sim \mathcal{N}(0, 1)$ and $\sigma \sim \text{Uniform}(0, 5)$.



The network (orange) matches MCMC (blue): it has **learned the posterior-mean map**. *Why?* — the loss it was trained with.

The loss picks which summary you estimate

The network minimises the **Bayes risk** $L(\gamma) = \mathbb{E}_{\theta, \mathbf{y}}[\ell(\theta, \hat{\theta}_\gamma(\mathbf{y}))]$ — and the loss ℓ decides *which* posterior summary it targets:

loss $\ell(\theta, \hat{\theta})$	best estimate (its minimiser)
squared $\ \theta - \hat{\theta}\ ^2$	posterior mean $\mathbb{E}[\theta \mathbf{y}]$
absolute $ \theta - \hat{\theta} $	posterior median
quantile $(\alpha - \mathbf{1}_{\{\theta < \hat{\theta}\}})(\theta - \hat{\theta})$	posterior quantile

Bayes risk is not available in closed form, so we *estimate it by Monte Carlo*:

$$L(\gamma) \approx \frac{1}{N} \sum_{i=1}^N \ell(\theta_i, \hat{\theta}_\gamma(\mathbf{y}_i)), \quad (\theta_i, \mathbf{y}_i) \sim \text{model}.$$

The idea

In training we estimate the risk by **Monte Carlo**:

$$L(\gamma) \approx \frac{1}{N} \sum_{i=1}^N \ell(\boldsymbol{\theta}_i, \hat{\boldsymbol{\theta}}_{\gamma}(\mathbf{y}_i)), \quad (\boldsymbol{\theta}_i, \mathbf{y}_i) \sim \text{model}.$$

- So the **gradient** $g(\gamma) = \nabla_{\gamma} L(\gamma)$ that drives training is a **random quantity with variance**.
- That variance is about *how we sample the loss*, not the inference problem — **so we can reduce it**.

Idea 1: Rao–Blackwellisation

Analytical conditional posteriors

- Split the parameters $\theta = (\theta_1, \theta_2)$. For many models, *conditioning* on θ_2 leaves the posterior of θ_1 in **closed form**:

$$p(\theta_1 \mid \theta_2, \mathbf{y}) \text{ known} \implies \mathbb{E}[\theta_1 \mid \theta_2, \mathbf{y}] \text{ is analytic.}$$

- This is **conditional conjugacy** — the same structure that makes *Gibbs sampling* possible.
- So: don't sample θ_1 — integrate it out using its exact conditional mean.

Example — Bayesian linear regression $\mathbf{y} = X\beta + \epsilon$: given the noise variance σ^2 , the coefficients β are Gaussian, so $\mathbb{E}[\beta \mid \sigma^2, \mathbf{y}]$ is closed-form ($\theta_1 = \beta$, $\theta_2 = \sigma^2$).

Expanding the Bayes risk

Under squared error, expand the risk by conditioning on $(\theta_2, \mathbf{y})$ ($\hat{\theta}_1$ = the network's estimate of θ_1):

$$\mathbb{E}[\|\theta_1 - \hat{\theta}_1\|^2 \mid \theta_2, \mathbf{y}] = \underbrace{\|\hat{\theta}_1 - \mathbb{E}[\theta_1 \mid \theta_2, \mathbf{y}]\|^2}_{\text{Rao-Blackwellised loss}} + \underbrace{\text{Var}(\theta_1 \mid \theta_2, \mathbf{y})}_{\text{constant in } \gamma}.$$

- The target for θ_1 becomes its **conditional posterior mean** $\mathbb{E}[\theta_1 \mid \theta_2, \mathbf{y}]$ — computed exactly, not sampled.
- The residual is free of γ : averaging over $(\theta_2, \mathbf{y})$, it is a **constant that drops out of the gradient**.
- So θ_1 is integrated out *analytically* instead of sampled.

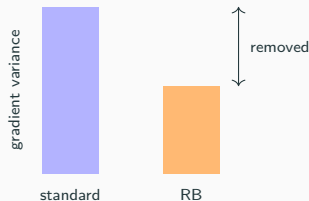
The gradient variance can only shrink

The Rao–Blackwellised gradient is the standard one **conditioned** on θ_2 :

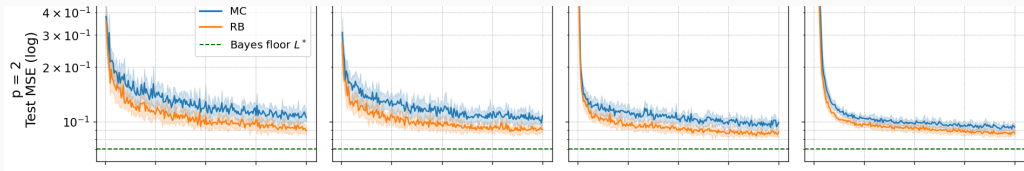
$g_{\text{RB}} = \mathbb{E}[g | \theta_2, \mathbf{y}]$. The law of total variance gives

$$\text{Var}(g) = \underbrace{\text{Var}(\mathbb{E}[g | \theta_2, \mathbf{y}])}_{\text{Var}(g_{\text{RB}})} + \underbrace{\mathbb{E}[\text{Var}(g | \theta_2, \mathbf{y})]}_{\succeq 0}.$$

- Hence $\text{Var}(g_{\text{RB}}) \preceq \text{Var}(g)$ — **never larger**, for any model or architecture.
- For a linear network the reduction reaches the fitted weights too.



Does it pay off? Bayesian linear regression



MC (blue) vs RB (orange) vs Bayes floor L^* (green) — across batch sizes 4, 16, 64, 256.

lower loss in
every grid cell

closes $\sim$ **half** (52%)
of the gap to L^*

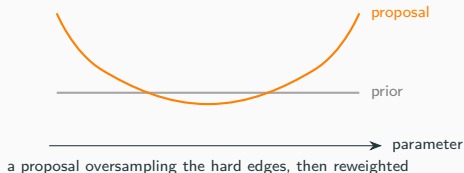
matches the standard NBE
at $\frac{1}{4}$ **the batch size**

Idea 2: importance sampling

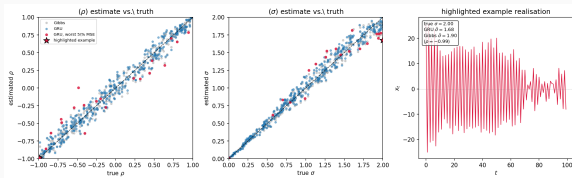
A more general lever

No conjugacy required — unlike Rao–Blackwell.

- Simulate from a **proposal** that oversamples the informative regions, then **reweight** back to the original risk (unbiased for any proposal).
- But **no guarantee**: bad weights can *increase* variance and shrink the effective sample size.

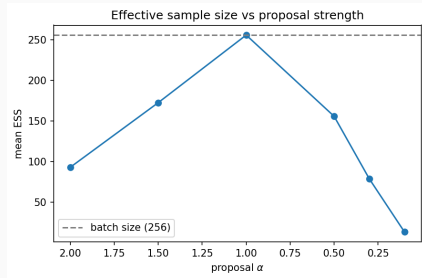


Importance sampling: the result



On an AR(1) model, the GRU already tracks the Gibbs (Bayes) reference across the prior — **little error left to recover.**

Why it fails here: the estimator already sits near the Bayes floor, and the little error that remains lives at the $|\rho| \rightarrow 1$ boundary — which the proposals do not target.



Push the proposal hard and the effective sample size **collapses** — only adding noise.

Conclusion

Conclusion

Training a neural Bayes estimator is itself Monte Carlo — so its training variance can be reduced.

- **Rao–Blackwellisation** (needs a conjugate block): a *provable* reduction — closes $\sim$ half the gap to the Bayes floor and matches the standard estimator at $\frac{1}{4}$ the batch size.
- **Importance sampling** (needs nothing special): more general, but unguaranteed and problem-dependent — here it did not help.

Conjugacy — the classical trick behind Gibbs sampling — is just as useful for *training* the estimators meant to replace it.

Thank you